

FastRobots-2026

ECE4160/5160-MAE 4190/5190: Fast Robots course, offered at Cornell University in Spring 2026

View On
GitHub

This project is maintained by FastRobotsCornell

Fast Robots @ Cornell

[Return to main page](#)

Lab 11: Localization on the real robot

Objective

In this lab you will perform localization with the Bayes filter on your actual robot. Note that we will only use the update step based on full 360 degree scans with the ToF sensor, because the motion of these particular robots is typically so noisy that it does not help us to do the prediction step. The point of the lab is to appreciate the difference between simulation and real-world systems. We will provide you with an optimized version of the localization code.

Parts Required

- 1 x Fully assembled [robot](#), with [Artemis](#), [TOF sensors](#), and an [IMU](#).

Prelab

Lectures

Consider going through the lectures on [Sensor Models](#), [Motion models](#), and the [Bayes Filter](#).

Grid Localization

Please refer to the [simulation setup](#) and [Lab 10](#).

Localization Module:

We provide you with a fully-functional and optimized Bayes filter implementation that works on the virtual robot. You will make changes in the code so that the module can work with your real robot. Note the time it takes to run the prediction and update step. You will find that we took several measures to enable fast computation in practice:

- Pre-caching ray casting values
- Numpy vector processing
- Skipping grid cells with low probabilities in the prediction step
- Utilizing fixed size numpy arrays instead of variable-length lists
- Minimizing the use of temporary variables

Ground Truth

Since there is no automated method to get the real robot's ground truth pose; you will need to identify certain known poses. You can measure the distance from the origin (0,0) of the map in terms of the number of (1 feet long) tiles in the x and y directions. For this lab, you will need to place your robot in the 4 marked positions in the map.

Observation Loop

Your robot needs to output ToF sensor readings taken at 20 degree increments, starting from the robot's heading. You can use both your ToF sensors to collectively output sensor range readings at 20 degree increments (i.e. +0, +20, +40 ,... +340 degrees from the current robot heading). Positive angles are along the counter-clockwise direction.

*If you are unable to get 18 sensor readings that are 20 degrees apart, you can change the number of sensor readings per loop (`observation_count`) in **world.yaml**. Remember that reducing the number of observations reduces the accuracy of your localized pose.*

Setup the base code

1. Copy `lab11_sim.ipynb` and `lab11_real.ipynb` from [here](#) into the **notebooks** directory (inside the simulation base code directory).
2. Copy `localization_extras.py` from [here](#) into the **root** directory (inside the **FastRobots-sim-release** directory).
3. You will need to copy the necessary Bluetooth python modules (present inside directory **ble_python**) from your previous labs. Copy **ONLY** the files “`base_ble.py`”, “`ble.py`”, “`connection.yaml`” and “`cmd_types.py`” into the **notebooks** directory.

Do not copy the `utils.py` file.

Files Provided:

1. **localization_extras.py**: Provides a fully-functional Localization module that works on the virtual robot.
2. **lab11_sim.ipynb**: A Jupyter notebook that demonstrates the Bayes filter implementation on the virtual robot.
3. **lab11_real.ipynb**: A Jupyter notebook that provides you with a skeleton code to integrate the real robot with the localization code.

Tasks

1. Test Localization in Simulation: Run the notebook **lab11_sim.ipynb** and attach a single screenshot of the final plot (odom, ground truth and belief).
2. Using a uniform prior on the pose, run (only) the update step using the sensor measurement data to localize your robot
 1. Go through the notebook **lab11_real.ipynb** and implement the member function **perform_observation_loop** of class **RealRobot** (re-use code from previous labs to implement this).

2. Place your robot in one of the four marked poses and run the update step of the Bayes filter once.
 - How close is the localized pose w.r.t to the ground truth?
 - Visualize your results
 - Discuss your results
3. Repeat (2) for every marked position.
 - Does the robot localize better in certain poses? If so, why?

Marked Poses

The four marked poses in the lab are:

- (-3 ft , -2 ft , 0 deg)
- (0 ft, 3 ft, 0 deg)
- (5 ft, -3 ft, 0 deg)
- (5 ft, 3 ft, 0 deg)

Tips

1. The output of the member function **perform_observation_loop()** needs to be a numpy column array, as required by the provided localization code. If you have a python list of the 18 sensor readings, use the code snippet below to convert the list to a numpy column array:

```
np.array(array)[np.newaxis].T
```

2. If you get poor localization results:
 1. Make sure the robot is performing a counter-clockwise rotation with the first sensor reading taken at the current robot heading.
 2. Adjust the sensor noise in the configuration file (sensor_sigma in sensor_sigma).
3. If you need to call asyncio coroutines (ex: `await asyncio.sleep(3)`) inside a function, you need to add the keyword `async` to the function definition. Here is an example:

```
async def sleep_for_3_secs():  
    await asyncio.sleep(3)
```

```
await sleep_for_3_secs()
```

NOTE: Notice the keyword “async” in the function definition. Any function that uses the keyword “await” (i.e. calls `asyncio.coroutine`), should have the `async` keyword in the definition. To call the `async` “function” or coroutine, use the “await” keyword.

4. If you need to use an `await` inside `RealRobot.perform_observation_loop()` for some reason (for e.g. you are using BLE handlers to get the observation data from the real robot), you need to make changes to the localization functions which call this function. Here is a walkthrough of the function definitions that need to be modified:

1. Add the `async` keyword to the function definition

```
RealRobot.perform_observation_loop()
```

2. Add the `async` keyword to the function definition

```
BaseLocalization.get_observation_data() [Ref]
```

3. Add the `await` keyword when calling the `async` coroutine

```
perform_observation_loop() [Ref]
```

4. In the code cell in the Jupyter notebook `lab11_real.ipynb`, append the `await` keyword to the line `loc.get_observation_data()`

NOTE: You could skip the above steps and instead directly call the `asyncio.sleep` coroutine as `asyncio.run(asyncio.sleep(3))` inside the non-`async` function `RealRobot.perform_observation_loop()`, however, this may not be the right way to use `asyncio` coroutines and could pose issues.

Write-up

To demonstrate that you’ve successfully completed the lab, please upload a brief lab report (<1000 words), with code (not included in the word count), photos, and videos documenting that everything worked and what you did to make it happen. Include the robot’s belief after localization for each pose, compare it with the ground truth pose, and write down your inference.

Hosted on

[GitHub Pages](#)

using the Dinky theme